

ภาคผนวก

ภาคผนวก ก

การใช้งาน Microsoft SQL Server 2000 driver for JDBC

การติดต่อกับฐานข้อมูล(Connecting to a Database)

เมื่อมีการติดตั้งไดรเวอร์ (driver) แล้วคุณก็สามารถติดต่อกับฐานข้อมูลของคุณได้จากแอปพลิเคชันของคุณ ซึ่งมีด้วยกัน 2 วิธีคือ การใช้ connection URL ผ่าน JDBC driver manager หรือ การใช้ JNDI data source

คุณสามารถติดต่อผ่าน JDBC driver manager ได้โดยการใช้เมธอด (method) DriverManager.getConnection เมธอดนี้ใช้สตริง (string) ในการเก็บค่าของ URL ใช้ขั้นตอนต่อไปนีในการโหลด (load) ไดรเวอร์จาก JDBC application ของคุณ

1. กำหนดคลาสพาธ (Setting the Classpath)

SQL Server 2000 Driver สำหรับ JDBC จำเป็นจะต้องถูกกำหนดในตัวแปร

CLASSPATH ของคุณ CLASSPATH คือ เซิร์ชสตริง (search string) ที่ Java Virtual Machine (JVM) ใช้ในการหาที่ตั้งของ JDBC ไดรเวอร์บนเครื่องคอมพิวเตอร์ของคุณ ถ้าไดรเวอร์ไม่ได้อยู่บน CLASSPATH นั้นคุณก็จะได้รับข้อความแสดงความผิดพลาด(error) ว่า “class not found” เมื่อมีการพยายามที่จะโหลดไดรเวอร์ขึ้นมา การกำหนด CLASSPATH ให้กับระบบของคุณนั้นรวมถึงเส้นทางต่อไปนี้ด้วย เมื่อ *install_dir* เป็นเส้นทาง (path) ที่จะไปยัง SQL Server 2000 Driver สำหรับ JDBC installation directory :

install_dir/lib/msbase.jar

install_dir/lib/msutil.jar

install_dir/lib/mssqlserver.jar

ตัวอย่างสำหรับวินโดวส์ (Windows)

CLASSPATH=.;c:\Microsoft SQL Server 2000 Driver for JDBC\lib\msbase.jar;c:\Microsoft SQL Server 2000 Driver for JDBC\lib\msutil.jar;c:\Microsoft SQL Server 2000 Driver for JDBC\lib\mssqlserver.jar

ตัวอย่างสำหรับยูนิกซ์ (UNIX)

CLASSPATH=./home/user1/mssqlserver2000jdbc/lib/msbase.jar;/home/user1/mssqlserver2000jdbc/lib/msutil.jar;/home/user1/mssqlserver2000jdbc/lib/mssqlserver.jar

2. การลงทะเบียนไดรเวอร์ (Registering the Driver)

การลงทะเบียนไดรเวอร์เป็นการบอก JDBC driver manager ว่าจะโหลดไดรเวอร์ไหน เมื่อมีการโหลดไดรเวอร์โดยใช้ class.forName() คุณจะต้องระบุชื่อของไดรเวอร์ดังนี้

com.microsoft.jdbc.sqlserver.SQLServerDriver

ตัวอย่างเช่น

```
Class.forName("com.microsoft.jdbc.sqlserver.SQLServerDriver");
```

3. Passing the Connection URL

หลังจากลงทะเบียนไดรเวอร์เรียบร้อยแล้ว คุณจำเป็นต้องส่งผ่านข้อมูลในการติดต่อฐานข้อมูลในรูปแบบของ connection URL และตามด้วย template URL สำหรับ SQL Server 2000 driver สำหรับ JDBC เป็นตัวแทนค่าในการระบุถึงฐานข้อมูลของคุณ jdbc:microsoft:sqlserver://server_name:1433

ตัวอย่างเช่นในการระบุ connection URL ซึ่งรวมถึง user ID “username” และพาสเวิร์ด “secret” กระทำได้ดังนี้

```
Connection conn = DriverManager.getConnection
```

```
("jdbc:microsoft:sqlserver://server1:1433","username","secret");
```

หมายเหตุ :

server_name คือ ไอพีแอดเดรสหรือชื่อโฮส(host name) เมื่อสมมติว่าเครือข่ายของคุณแยกชื่อโฮสออกจากไอพีแอดเดรส คุณสามารถทดสอบสิ่งนี้ได้จากการใช้คำสั่ง ping ในการเข้าถึง (access) ชื่อโฮสและตรวจสอบว่าคุณได้รับการตอบกลับจากไอพีแอดเดรสที่ถูกต้อง

ค่าทางตัวเลข (numeric value) ที่ตามหลังชื่อเซิร์ฟเวอร์ คือหมายเลขพอร์ตที่ฐานข้อมูลกำลังรอการติดต่อมาจากไคลเอนท์ (listen) ค่าที่แสดงในตอนนี้เป็นค่าเริ่มต้น (default) คุณควรที่จะเลือกหมายเลขพอร์ตที่จะให้ฐานข้อมูลของคุณใช้และใส่แทนที่ค่าดังกล่าว

การใช้ SQL Server 2000 ไดรเวอร์สำหรับ JDBC (Using the SQL Server 2000 Driver for JDBC)

Type 4 SQL Server 2000 ไดรเวอร์สำหรับ JDBC นั้นรองรับการเข้าถึงของ JDBC ผ่านไปยังทุก Java-enabled แอปเพลต,แอปพพลิเคชัน หรือ แอปพพลิเคชันเซิร์ฟเวอร์ ซึ่งมีประสิทธิภาพสูงในการเข้าถึง Microsoft SQL Server ข้ามInternet และ intranets ทั้งแบบจุดต่อจุด (point-to-point) และ n-tier ไดรเวอร์จะมีประสิทธิภาพสูงที่สุดกับ Java environment คือการอนุญาตให้คุณรวมเอาเทคโนโลยีของ Java แล้วเพิ่ม functionality และประสิทธิภาพของระบบที่คุณมีอยู่

เกี่ยวกับ SQL Server 2000 ไดรเวอร์สำหรับ JDBC (About the SQL Server 2000 Driver for JDBC)

SQL Server 2000 Driver สำหรับ JDBC นั้นรองรับ JDBC 2.0 specification นอกจากนี้ไดรเวอร์ยังรองรับ subset ของ JDBC 2.0 Optional Package อีกด้วย ซึ่งประกอบด้วย

- 1 Java Naming Directory Interface (JNDI) สำหรับ naming data source
- 2 Connection Pooling

การติดต่อผ่าน JDBC Driver Manager (Connecting through the JDBC Driver Manager)

วิธีการหนึ่งที่ทำให้เราสามารถติดต่อกับฐานข้อมูลได้คือการติดต่อผ่านทาง JDBC driver manager โดยการใช้เมธอด DriverManager.getConnection เมธอดนี้ใช้สตริงในการเก็บค่าของ URL ตัวอย่างต่อไปนี้จะแสดงการใช้งาน JDBC driver manager ในการติดต่อกับ Microsoft SQL Server 2000 โดยมีการส่งผ่านค่า username และ password ไปให้

```
Class.forName("com.microsoft.jdbc.sqlserver.SQLServerDriver");
Connection conn = DriverManager.getConnection
("jdbc:microsoft:sqlserver://server1:1433;User=test;Password=secret");
```

ตัวอย่าง URL(URL Examples)

รูปแบบของ connection URL ที่สมบูรณ์ที่ใช้ใน driver manager เป็นดังนี้
 jdbc:microsoft:sqlserver://*hostname:port[:property=value...]* โดยที่
 hostname คือ TCP/IP address หรือ TCP/IP hostname ของ เซิร์ฟเวอร์ที่คุณใช้ในการติดต่อ
 หมายเหตุ: untrusted applets ไม่สามารถที่จะเปิดช็อกเก็ตไปยังเครื่องได้เหมือนกับตัวโฮสเอง
 Port คือหมายเลขพอร์ต TCP/IP

property = value เป็นการกำหนดคุณสมบัติของการเชื่อมต่อ

ตัวอย่าง

```
jdbc:microsoft:sqlserver://server1:1433;user=test;password=secret
```

การติดต่อผ่าน Data Source (Connecting Through Data Source)

SQL Server 2000 Driver สำหรับ Data Source object มีข้อมูลที่เป็นต้องใช้ในการติดต่อกับ underlying database ประโยชน์หลักของการใช้ data source คือมันทำงานร่วมกับ Java Naming Directory Interface (JNDI) naming service และมันสามารถถูกสร้างและถูกจัดการจากภายนอกแอปพลิเคชันที่ใช้งาน มันอยู่ได้ เพราะว่าข้อมูลที่ใช้ในการติดต่อนั้นอยู่นอกแอปพลิเคชัน ทำให้เวลาที่ใช้ในการปรับเปลี่ยน infrastructure เมื่อมีการเปลี่ยนแปลงเกิดขึ้นนั้นน้อยที่สุด ตัวอย่างเช่น ถ้ามีการย้าย underlying database ไปยังอีกเซิร์ฟเวอร์หนึ่งและมีการเปลี่ยนหมายเลขพอร์ต ผู้ดูแลระบบ (Administrator) ก็จะต้องแก้ไข คุณสมบัติที่สัมพันธ์กันของ SQL Server 2000 driver สำหรับ JDBC data source (DataSource object) แอปพลิเคชันที่ใช้งาน underlying database ดังกล่าวไม่จำเป็นที่จะต้องแก้ไขเปลี่ยนแปลงอะไร เพราะว่า เขาส่งแค่ logical name ของ SQL Server 2000 Driver สำหรับ JDBC data source มาเท่านั้น

เราจะimplement SQL Server 2000 Driver สำหรับ JDBC Data Sources ได้อย่างไร

Microsoft ได้นำเสนอ data source class สำหรับ SQL Server 2000 Driver สำหรับ JDBC data source ไว้ด้วย ชื่อของ class ได้ที่ SQL Server 2000 Driver for JDBC

SQL Server 2000 Driver สำหรับ JDBC data source class มีการ implement interfaces ใน JDBC 2.0 Optional Package ดังนี้

- 1 javax.sql.DataSource
- 2 javax.sql.ConnectionPoolDataSource ซึ่งทำให้คุณสามารถที่จะ implement connection pooling ได้

หมายเหตุ : คุณควรจะนำเอา class javax.sql.* และ javax.naming.* เข้ามาเพื่อที่จะสร้างและใช้งาน SQL Server 2000 Driver สำหรับ JDBC data sources นอกจากนี้แล้ว SQL Server 2000 Driver สำหรับ JDBC ยังมี JAR files ที่จำเป็นที่ประกอบด้วย classes และ interfaces ต่างๆ ที่ต้องการ

การเรียกใช้ Data Source ในแอปพลิเคชัน (Calling a Data Source in an Application)

แอปพลิเคชันสามารถเรียกใช้ SQL Server 2000 Driver สำหรับ JDBC data source โดยการใช้ logical name ในการ retrieve javax.sql.DataSource object โดยที่ object นี้จะโหลด SQL Server 2000 Driver สำหรับ JDBC แล้วสามารถถูกใช้ในการสร้าง (establish) การสื่อสารไปยัง underlying database

เมื่อ SQL Server 2000 Driver สำหรับ JDBC data source ถูกลงทะเบียน (registered) ด้วย JNDI มันสามารถถูกใช้งานโดย JDBC แอปพลิเคชันของคุณในการทำสิ่งต่างๆ ดังตัวอย่างต่อไปนี้

```
Context ctx = new InitialContext();
DataSource ds = (DataSource)ctx.lookup("jdbc/EmployeeDB");
Connection con = ds.getConnection("matt", "wwf");
```

จากตัวอย่างนี้จะเห็นได้ว่า JNDI Environment ถูก initialized ในตอนแรก จากนั้นมีการใช้ initial naming context ในการหา logical name ของ SQL Server 2000 Driver สำหรับ JDBC data source (EmployeeDB) เมธอด Context.lookup() ส่งค่าอ้างอิงกลับมายัง Java object ซึ่งจะจำกัดเฉพาะ javax.sql.DataSource object สุดท้ายเมธอด DataSource.getConnection() ถูกเรียกขึ้นมาเพื่อสร้างการเชื่อมต่อกับ underlying database

การใช้ connection pooling (Using Connection Pooling)

Connection pooling สนับสนุนให้คุณทำการ reuse connections มากกว่าการสร้างขึ้นมาใหม่ทุกครั้ง SQL Server 2000 Driver สำหรับ JDBC จำเป็นต้องติดต่อสื่อสารกับ underlying database Connection pooling จัดการ connection sharing ระหว่าง user request ต่างๆ เพื่อรักษาประสิทธิภาพและลดจำนวน connection ใหม่ที่จะถูกสร้างขึ้น ตัวอย่าง เปรียบเทียบ transaction sequence ต่อไปนี้

A: ไม่มี connection pooling

1. ไคลเอนต์แอปพลิเคชัน (client application) สร้าง connection

2. ไคลเอนต์แอปพลิเคชันส่ง data access query
3. ไคลเอนต์แอปพลิเคชันรับ result set ของการ query
4. ไคลเอนต์แอปพลิเคชันแสดง result set ให้กับ end user
5. ไคลเอนต์แอปพลิเคชันจบการสื่อสาร

B: มี connection polling

1. client ตรวจสอบ connection poll เพื่อหา connection ที่ไม่ได้ใช้
2. ถ้ามี connection ที่ไม่ได้ใช้อยู่ connection นั้นก็จะถูก returned กลับมาโดย pool implementation ในทางตรงกันข้าม เมื่อไม่มีก็สร้าง connection ขึ้นมาใหม่
3. ไคลเอนต์แอปพลิเคชันส่ง data access query
4. ไคลเอนต์แอปพลิเคชันรับ result set ของการ query
5. ไคลเอนต์แอปพลิเคชันแสดง result set ที่ได้ให้กับ end user
6. ไคลเอนต์แอปพลิเคชัน return connection กลับให้กับ pool

หมายเหตุ: ไคลเอนต์แอปพลิเคชันยังคงเรียกเมธอด close() แต่ connection ยังคงเปิดอยู่และ pool เองก็รับทราบเมื่อมีการเรียกใช้เมธอดนี้

Pool implementation สร้างการติดต่อกับฐานข้อมูลอย่างแท้จริง (“real” database connection) โดยใช้เมธอดของ ConnectionPoolDataSource คือเมธอด getPooledConnection() จากนั้น pool implementation ก็จัดการตัวเองให้เป็น Listener ของ PooledConnection เมื่อไคลเอนต์แอปพลิเคชันร้องขอการเชื่อมต่อ pool implementation (Pool Manager) ก็จะกำหนด connection ของมันที่มีอยู่อันใดอันหนึ่งให้ มิฉะนั้นไม่มี connection อยู่เลย Pool Manager ก็จะสร้างขึ้นมาให้ใหม่และกำหนด connection นี้ให้กับแอปพลิเคชันนั้น เมื่อไคลเอนต์แอปพลิเคชันบอกยกเลิกการเชื่อมต่อ (close) Pool Manager ก็จะรับทราบได้จากไครเวอร์ผ่าน ConnectionEventListener interface ว่า connection นั้นว่างแล้วและพร้อมที่จะให้ reuse ส่วน pool implementation เองก็รับทราบได้จาก ConnectionEventListener interface เช่นกันเมื่อไคลเอนต์เกิดความผิดพลาดในการเชื่อมต่อกับฐานข้อมูลไม่ว่าด้วยเหตุผลใดก็ตาม ซึ่งทำให้ pool implementation สามารถที่จะเอา connection นั้นออกจาก pool ได้

เมื่อ SQL Server 2000 Driver สำหรับ JDBC data source ถูกลงทะเบียนด้วย JNDI มันก็สามารถถูกใช้งานได้โดย JDBC แอปพลิเคชันดังที่จะแสดงในตัวอย่างต่อไปนี้ ซึ่งใช้งานผ่านทาง third party connection pool tool:

```
Context ctx = new InitialContext();
ConnectionPoolDataSource ds =
```

```
(ConnectionPoolDataSource)ctx.lookup("jdbc/EmployeeDB");
pooledConnection pcon = ds.getPooledConnection("matt", "wwf");
```

จากตัวอย่างนี้ JNDI Environment ถูก initialized ในตอนแรก จากนั้นมีการใช้ initial naming context ในการหา logical name ของ SQL Server 2000 Driver สำหรับ JDBC data source (EmployeeDB) เมธอด Context.lookup() return การอ้างอิงกลับมายัง Java object ซึ่งจะจำกัดเฉพาะ javax.sql.ConnectionPoolDataSourceobject สุดท้ายเมธอด ConnectionPoolDataSource.getPooledConnection() ถูกเรียกขึ้นมาเพื่อสร้างการเชื่อมต่อกับ underlying database

หมายเหตุ: JDBC ไดรเวอร์ไม่ได้จัดการ connection pooling คุณจะต้องใช้ external connection pool manager มาช่วย

การใช้ SQL Server 2000 Driver สำหรับ JDBC บน Java 2 แพลตฟอร์ม

เมื่อมีการใช้ SQL Server 2000 Driver สำหรับ JDBC บน Java 2 แพลตฟอร์มกับตัวจัดการมาตรฐานความปลอดภัย (standard security manager) คุณจำเป็นต้องเพิ่ม permission บางอย่างให้กับ ไดรเวอร์ โดยอ้างอิงกับข้อมูลเกี่ยวกับ Java 2 platform security model and permission ในเอกสาร Java 2 แพลตฟอร์มของคุณ

คุณสามารถรันแอปพลิเคชันบน Java 2 แพลตฟอร์มกับตัวจัดการมาตรฐานความปลอดภัย (standard security manager) โดยการใช้

```
"java -Djava.security.manager application_class_name"
```

เมื่อ *application_class_name* คือชื่อของคลาสของแอปพลิเคชัน

เว็บเบราว์เซอร์แอปพลิเคชันที่รันใน Java 2 plug-in นั้นจะต้องรันใน Java Virtual Machine ที่มีตัวจัดการมาตรฐานความปลอดภัย (standard security manager) เสมอ ในการให้permissions ต่างๆที่จำเป็น คุณจะต้องเพิ่มมันเข้าไปในไฟล์ security policy ของ Java 2 แพลตฟอร์ม ซึ่งไฟล์นี้จะอยู่ที่ subdirectory jre/lib/security ใน installation directory ของ Java 2 แพลตฟอร์ม

ในการใช้ JDBC data sources ทุก code bases จะต้องเป็นไปตาม permission ต่อไปนี้

```
// permissions granted to all domains
grant {

// DataSource access

permission java.util.PropertyPermission "java.naming.*", "read,write";

// Adjust the server host specification for your environment

permission java.net.SocketPermission "*.microsoft.com:0-65535", "connect";

};
```

การที่จะใช้ insensitive scrollable cursors และในการปฏิบัติการจัดเรียง Database MetaData Resultsets ในฝั่งไคลเอนทั้นั้นทุก code bases จะต้องเข้าถึง (access) ไฟล์ temporary

สำหรับ JDK 1.1 environments จะต้องมีการยอมให้เข้าถึงไดเรกทอรีที่กำลังทำงานอยู่ ณ ปัจจุบัน

(current working directory)

สำหรับ Java 2 environments จะต้องมีการยอมให้เข้าถึง temporary directory ที่กำหนดโดย VM configuration

ตัวอย่างของ permission ที่ให้แก่ C:\TEMP ไดเรกทอรี

```
// permissions granted to all domains
grant {
// Permission to create and delete temporary files.
// Adjust the temporary directory for your environment.
permission java.io.FilePermission "C:\\TEMP\\-", "read,write,delete";
};
```

การจัดการกับความผิดพลาด(Error Handling)

SQL Server 2000 Driver สำหรับ JDBC นั้นรายงานความผิดพลาดให้กับแอปพลิเคชันที่เรียกใช้งานมันด้วยการ throwing SQLExceptions โดยที่แต่ละ SQLException มีข้อมูลดังต่อไปนี้

1. ข้อความที่อธิบายถึงสาเหตุที่เป็นไปได้ที่ทำให้เกิดข้อผิดพลาด ซึ่งระบุโดยคอมโพเนนท์ที่ทำให้เกิดข้อผิดพลาด
2. โคลด์ที่เป็นตัวที่ทำให้เกิดความผิดพลาด (ถ้าทำได้)
3. สตริงที่เก็บ XOPEN SQLState

SQL Server 2000 Driver For JDBC Errors

ความผิดพลาดที่เกิดขึ้นจาก SQL Server 2000 Driver สำหรับ JDBC มีลักษณะดังต่อไปนี้

[Microsoft][SQL Server 2000 Driver for JDBC]message

ตัวอย่างเช่น

[Microsoft][SQL Server 2000 Driver for JDBC]Timeout expired.

เมื่อเกิดข้อผิดพลาดเช่นนี้คุณอาจจำเป็นต้องตรวจสอบครั้งสุดท้ายที่ JDBC มีการเรียกแอปพลิเคชันของคุณ โดยตรวจสอบกับJDBC specification เกี่ยวกับ recommended action

Database Errors

ความผิดพลาดที่เกิดจากฐานข้อมูลมีลักษณะดังนี้

[Microsoft][SQL Server 2000 Driver for JDBC][SQL Server] message

ตัวอย่างเช่น

[Microsoft][SQL Server 2000 Driver for JDBC][SQL Server]

Invalid Object Name.

ใช้โค้ดที่เป็นตัวที่ทำให้เกิดความผิดพลาดมาตรวจสอบรายละเอียดเกี่ยวกับสาเหตุที่เป็นไปได้ที่ทำให้เกิดข้อผิดพลาด ซึ่งรายละเอียดนั้นสามารถดูได้จากเอกสารประกอบของฐานข้อมูล

โครงสร้างของไดเรกทอรีของSQL Server 2000 Driver For JDBC

ตารางที่ ก-1 แสดงถึงโครงสร้างของไดเรกทอรีของ SQL Server 2000 Driver สำหรับ JDBC และมีคำอธิบายเกี่ยวกับไฟล์และไดเรกทอรี ซึ่งไฟล์และไดเรกทอรีทั้งหมดนั้นอยู่ใน SQL Server 2000 Driver สำหรับ JDBC installation directory

Directories and Files	Description
uninstall.class	Executable that uninstalls the SQL Server 2000 Driver for JDBC (Windows only).
/books/	Directory that contains all of the SQL Server 2000 Driver for JDBC online documentation.
/Help/	Directory that contains the SQL Server 2000 Driver for JDBC online help.
/lib/mssqlserver.jar	Jar file containing the SQL Server 2000 Driver for JDBC and data source classes, specifically: com.microsoft.jdbc.sqlserver.SQLServerDriver and com.microsoft.jdbcx.sqlserver.SQLServerDataSource as well as other SQL Server driver-specific classes. This Jar file must be on your CLASSPATH to use the SQL Server 2000 Driver for JDBC.
/lib/msbase.jar	Jar file containing classes that are used by the SQL Server 2000 Driver for JDBC. This Jar file must be on your CLASSPATH to use the SQL Server 2000 Driver for JDBC.
/lib/msutil.jar	Jar file containing classes that are used by the SQL Server 2000 Driver for JDBC. This Jar file must be on your CLASSPATH to use the SQL Server 2000 Driver for JDBC.
/SQLServer JTA/instjdbc.sql /SQLServer JTA/sqljdbc.dll	Files used for installing JTA stored procedures for SQL

ตาราง ก-1 SQL Server 2000 Driver for JDBC Directory and Files

SQL Server 2000 Driver For JDBC

SQL Server 2000 Driver For JDBC (the “SQL Server Driver”) นั้นรองรับระบบฐานข้อมูล SQL Server 2000 ของ Microsoft

ในการใช้ JDBC distributed transactions ผ่าน JTA คุณจำเป็นต้องติดตั้ง stored procedure สำหรับ SQL Server

หมายเหตุ: ถึงแม้ว่า SQL Server driver จะรองรับ JTA แต่จะไม่สามารถใช้ร่วมกับเมธอด SelectMethod ได้โดยตรง

Data Source และ Driver Classes

datasourceclass สำหรับ SQL Server driver คือ

com.microsoft.jdbc.sqlserver.SQLServerDataSource

driver class สำหรับ SQL Server คือ com.microsoft.jdbc.sqlserver.SQLServerDriver

คุณสมบัติของ Connection String (Connection String Properties)

คุณสามารถใช้คุณสมบัติของ connection ต่อไปนี้กับ JDBC driver manager หรือ SQL Server 2000 driver สำหรับ JDBC data source

ตารางที่ ก-2 แสดงถึงคุณสมบัติของ JDBC connection ที่รองรับโดย SQL Server driver และอธิบายถึงรายละเอียดของคุณสมบัติแต่ละอัน ซึ่งคุณสมบัตินั้นอยู่ในรูปของ *property=value* หมายเหตุ : ทุกคุณสมบัติของ Connection String นั้นเป็น case-insensitive ตัวอย่างเช่น PortNumber นั้นก็เหมือนกับ portnumber

Property	Description
DatabaseName OPTIONAL	The name of the SQL Server database to which you want to connect.
HostProcess OPTIONAL	The process ID of the application connecting to SQL Server 2000. The supplied value appears in the "hostprocess" column of the sysprocesses table.
NetAddress OPTIONAL	The MAC address of the network interface card of the application connecting to SQL Server 2000. The supplied value appears in the "net_address" column of the sysprocesses table.
Password	The case-insensitive password used to connect to your SQL Server database.
PortNumber OPTIONAL	The TCP port (use for DataSource connections only). The default is 1433.
ProgramName OPTIONAL	The name of the application connecting to SQL Server 2000. The supplied value appears in the "program_name" column of the sysprocesses table.
SelectMethod	SelectMethod={cursor direct}. Determines whether database cursors are used for Select statements. Performance and behavior of the driver are

	affected by the SelectMethod setting. Direct-The direct method sends the complete result set in one request to the driver. It is useful for queries that only produce a small amount of data that you fetch completely. You should avoid using direct when executing queries that produce a large amount of data, as the result set is cached completely on the client and constrains memory. In this mode, each statement requires its own connection to the database. This is accomplished by "cloning" connections. Cloned connections use the same connection properties as the original connection; however, because transactions must occur on a single connection, auto commit mode is required. Due to this, <i>JTA is not supported</i> in direct mode. In addition, some operations, such as updating an insensitive result set, are not supported in direct mode because the driver must create a second statement internally. Exceptions generated due to the creation of cloned statements usually return an error message similar to "Cannot start a cloned connection while in manual transaction mode." Cursor-When the SelectMethod is set to cursor, a server-side cursor is generated. The rows are fetched from the server in blocks. The JDBC Statement method setFetchSize can be used to control the number of rows that are fetched per request. The cursor method is useful for queries that produce a large amount of data, data that is too large to cache on the client. Performance tests show that the value of setFetchSize has a serious impact on performance when SelectMethod is set to cursor. There is no simple rule for determining the value that you should use. You should experiment with different setFetchSize values to find out which value gives the best performance for your application. The default is direct.
SendStringParametersAsUnicode	SendStringParametersAsUnicode={true false}. Determines whether string parameters are sent to the SQL Server database in Unicode or in the default character encoding of the database. True means that string parameters are sent to SQL Server in Unicode. False means that they are sent in the default encoding, which can improve performance because the server does not need to convert Unicode characters to the default encoding. You should, however, use default encoding only if the parameter string data that you specify is consistent with the default

	encoding of the database. The default is true.
ServerName	The IP address (use for DataSource connections only).
User	The case-insensitive user name used to connect to your SQL Server database.

ตารางที่ ๑๖-2 SQL Server Connection String Properties

Data Types

SQL Server Data Type	JDBC Data Type
bigint	BIGINT
bigint identity	BIGINT
binary	BINARY
bit	BIT
char	CHAR
datetime	TIMESTAMP
decimal	DECIMAL
decimal() identity	DECIMAL
float	FLOAT
image	LONGVARBINARY
int	INTEGER
int identity	INTEGER
money	DECIMAL
nchar	CHAR
ntext	LONGVARCHAR
numeric	NUMERIC
numeric() identity	NUMERIC
nvarchar	VARCHAR
real	REAL
smalldatetime	TIMESTAMP
smallint	SMALLINT
smallint identity	SMALLINT
smallmoney	DECIMAL
sql_variant	VARCHAR

sysname	VARCHAR
text	LONGVARCHAR
timestamp	BINARY
tinyint	TINYINT
tinyint identity	TINYINT
uniqueidentifier	CHAR
varbinary	VARBINARY
varchar	VARCHAR

ตารางที่ ก-3 แสดง *data types* ต่างๆที่รองรับ *SQL Server driver* และเทียบกับ *JDBC data types*

การแยกออกเป็นลำดับชั้น (Isolation Level)

SQL Server รองรับการแยกออกเป็นลำดับชั้นของ Read Committed, Read Uncommitted, Repeatable Read และ Serializable โดยมีค่าตั้งต้น (default) เป็น Read Committed

Using Scrollable Cursors

SQL Server driver นั้นรองรับ scroll-sensitive result sets และ updatable result sets

หมายเหตุ : เมื่อ SQL Server driver ไม่สามารถรองรับ result set type หรือ ทำพร้อมกันตามที่ถูกร้องขอได้ มันก็จะลดระดับของเคอร์เซอร์ลงโดยอัตโนมัติและสร้าง SQL Warnings ที่มีรายละเอียดอยู่ด้วยขึ้นมา

ภาคผนวก ข

แอปพลิเคชันโปรแกรมมิ่งอินเทอร์เฟซ (Application programming interface (API)) ที่ใช้ในการพัฒนาระบบจัดการอุปกรณ์ผ่านเครือข่าย

1. เครื่องมือในการพัฒนาการสื่อสารบนเครือข่ายอินเทอร์เน็ต IP*Works!

IP*Works! Internet Toolkit

IP*Works! is a comprehensive set of tools including everything you need to write powerful connected applications.

Initially introduced in 1995, IP*Works! is actively used by most Fortune 500 and Global 2000 companies, as well as thousands of independent [software developers worldwide](#). The product eliminates much of the complexity of developing connected applications, by providing easy-to-use programmable components that facilitate tasks such as sending email, transferring files, browsing the web, consuming web services, parsing XML documents, etc..

IP*Works! is available in more than 20 editions, targeting various Platforms, IDEs, and component technologies.

Supported Platforms

Microsoft .NET

IP*Works! .NET Edition contains fully-managed .NET components based on a 100% C# codebase, with no dependencies in outside native code.

Windows (Desktop and Server)

IP*Works! supports all versions of Windows 95, Windows 98, Windows NT, Windows 2000, Windows Me, as well Windows XP (some special configuration steps are required for IP*Works! SSL in early versions of Windows).

IP*Works! ASP Edition and IP*Works! SSL ASP Edition are the editions of choice for Active Server Pages.

Windows CE (Pocket PC)

IP*Works! ActiveX Edition and IP*Works! C++ Edition currently support Pocket PC 2002, Pocket PC 2000, HPC 2000, and SmartPhone. These products provide the same interfaces and the same functionality as the corresponding desktop versions, but in an entirely different form factor.

Java

IP*Works! Java Edition is a Pure Java version of IP*Works! rewritten from the ground up. It has the same interfaces and same functionality as other editions of IP*Works!, provided in Pure Java classes for extreme portability across platforms, running on any standard JVM.

Linux (Native)

IP*Works! now has two versions native to the Linux platform. IP*Works! C++ Linux Edition and IP*Works! Kylix Edition are new editions to the IP*Works! family. These Linux versions share the same interfaces and support asynchronous sockets just like their Windows counterparts.

Solaris (Native)

IP*Works! C++ Edition for Solaris provides the same objects with the same interfaces as its Windows and Linux counterparts, including full support for asynchronous sockets.

Mac OS X (Native)

IP*Works! C++ Edition for Mac OS X brings unprecedented new functionality to the hands of Mac OS X developers facilitating the integration of Internet functionality within native Mac OS X applications.

Highlights

- The components are very easy to use, with a uniform, intuitive, and extensible design.
- Fast, robust, and reliable, the components consume a minimum of resources.
- We charge no runtime fees, and no royalties of any kind to application developers. Only one license for the development environment.
- The components are fully documented, with easy to use, fully indexed help files, and printed manuals. The help files are fully integrated in the IDE.
- The components offer easy to use implementation of almost all the popular Internet protocols, and the range of features they cover, means that you can build professional Internet/Intranet applications with little or no effort.
- Developed in pure Java for extreme portability across platforms

- An excellent 7+ year track record, maintained, supported and enhanced by a professional staff.

FileMailer - The FileMailer control provides an easy way to send emails with file attachments.

Rexec - Executes commands remotely on rexec-capable hosts: UNIX, mainframes, etc..

HTMLMailer - An easy way to send HTML Mail including embedded images.

Rshell - Executes commands remotely on rexec-capable hosts: UNIX, mainframes, etc..

NetClock - The NetClock control provides the current (GMT) time from an Internet Time Server.

SMTP - Used to send Internet (SMTP) Mail. Standards-based and extensible.

RCP - The RCP control implements Remote Copy Protocol, used to transfer files between systems.

SNMP - Create custom Network Management applications. Easy to use, powerful, standards-based.

FTP - Transfer files to and from FTP servers. Easy, plug and play, extensible interface.

SNPP - Send messages to alphanumeric pagers through standard Internet Paging Gateways.

HTTP - Access the World Wide Web from your applications, including HTML forms.

SOAP - Generic SOAP client supporting SOAP 1.1. Built on top of the IP*Works! XML parser.

IMAP - Easy access to corporate mail servers such as Microsoft Exchange.

Telnet - Telnet client component. Easy to use, standards-based, flexible, powerful.

IPInfo - A collection of DNS and other database functions. Both direct and reverse DNS queries.

TFTP - The TFTPClient control is used to exchange files with TFTP servers via the TFTP protocol.

IPPort - Generic TCP stream client component. Asynchronous architecture.

UDPPort - Easy interface to UDP packet communications. Use to build clients or servers.

LDAP - Search, manage, and maintain Internet Directory (LDAP) servers.

WebForm - The WebForm control is used to POST data to interactive web pages or scripts.

MCast - Internet Multicast component. Asynchronous architecture. Use to build

WebUpload - The WebUpload control is used to upload files to web servers.

clients or servers.

MX - Find email servers for a particular domain/address. Often used to verify email address input.

Whois - The Whois control allows you to query a WHOIS Server for Domain registration information.

NetCode - A collection of popular encoding and decoding functions: Base64, UUencode, for XPath DOM traversal and XML structure URL, etc..

XMLp - SAX2 compliant XML parser with support validation.

NNTP - Read, search, and post articles on Internet Newsgroups (USENET).

MIME - Easily encode and decode MIME structures such as message attachments, file uploads, etc..

POP - Easy retrieval of Internet Mail. Implements the standard POP3 protocol.

IPDaemon - Generic TCP server component. Asynchronous, event-driven architecture (not available for Pocket PC).

IP*Works! | [ASP](#)

The SNMP component is used to implement SNMP Management Applications and SNMP Agent Applications.

NOTE: What follows are very short descriptions of the control interfaces. For more information, please consult the help files that come with the respective package.

The SNMP component is used to implement SNMP Management Applications and SNMP Agent Applications. The SNMP component implements a standard SNMP Manager and/or Agent as specified in the SNMP RFCs. Both SNMP v1 and SNMP v2c are supported.

The component provides both encoding/decoding and transport capabilities, making the task of developing a custom SNMP agent or manager as simple as setting a few key properties and handling a few events. SNMP data, such as for instance SNMP object id-s (OID-s) are exchanged as text strings, thus further simplifying the task of handling them.

The component is activated/deactivated by first setting the **LocalPort** to 161 or 162, depending on whether you want to implement an SNMP agent or SNMP manager, and then setting the **Active** property. This property enables or disables sending or receiving. It operates completely

asynchronously. Messages are sent to other agents or managers by using component's methods or the **Action** property, and are received through events such as **GetRequest** , **GetResponse** , or **Trap** .

SNMP object ids, types, and values are provided in arrays such as **ObjId** , **ObjType** , and **ObjValue** , for both sent and received packets. **ObjCount** provides the number of elements in each of the arrays. Other packet information is provided through corresponding properties, such as **Community** , or **RequestId** , or similarly named parameters in events.

The component may behave as an SNMP agent or SNMP manager, depending on the value of the **LocalPort** property. If the component listens to port 161 it may act as an SNMP agent by responding to SNMP requests from SNMP managers, and sending traps through the **Action** property. If the **LocalPort** is set to 162, the component listens for SNMP traps, and may send requests to SNMP agents listening on port 161.

SNMP Traps are received through the **Trap** event, and may be sent through the **Action** property by specifying appropriate values in the various trap properties, such as **TrapAgentAddress** , **TrapEnterprise** , **TrapGenericType** , **TrapSpecificType** , and **TrapTimeStamp** .

Properties

AcceptData	Enables or disables data reception.
Action	An action code for the component.
Active	Enables or disables sending and receiving of SNMP packets.
Community	The community string used to authenticate SNMP packets.
ErrorIndex	Index of the first variable (object) that caused an error.
ErrorStatus	Status code for outgoing 'GET RESPONSE' packets.
InBufferSize	The size in bytes of the incoming queue of the socket.
LocalHost	The name of the local host or user-assigned IP interface through which connections are initiated or accepted.
LocalPort	The UDP port in the local host where the SNMP component listens to.
MaxRepetitions	Used by 'BULK REQUEST' packets.
NonRepeaters	Used by 'BULK REQUEST' packets.
ObjCount	Number of objects in the current request.

ObjId	Array of OIDs encoded as strings.
ObjType	Array of object types.
ObjValue	Array of object values.
OutBufferSize	The size in bytes of the outgoing queue of the socket.
RemoteHost	The address of the remote host. Domain names are resolved to IP addresses.
RemotePort	The UDP port where the remote SNMP agent is listening.
RequestId	The request-id to mark outgoing packets with.
ShareLocalPort	If set to True, allows more than one instance of the component to be Active on the same LocalPort .
SNMPVersion	Version of SNMP used for outgoing packets.
SocketHandle	The handle of the main socket used by the component.
TrapAgentAddress	The address of the object generating the trap.
TrapEnterprise	The type of the object generating the trap.
TrapGenericType	The generic type of the trap being sent.
TrapSpecificType	The specific type of the trap being sent.
TrapTimeStamp	Time passed since the agent was initialized (in hundredths of a second).
WinsockInfo	Identifying information about the loaded Winsock stack.
WinsockLoaded	Loads and unloads Winsock on demand.
WinsockMaxDatagramSize	Size in bytes of the largest UDP datagram that can be sent or received.
WinsockMaxSockets	Maximum number of sockets available to a single process.
WinsockPath	The path to the Winsock DLL used.
WinsockStatus	The status of the Winsock stack.

Events

Error	Information about errors during data delivery.
GetBulkRequest	Fired when a GetBulkRequest packet is received.
GetNextRequest	Fired when a GetNextRequest packet is received.

GetRequest	Fired when a GetRequest packet is received.
GetResponse	Fired when a GetResponse packet is received.
InformRequest	Fired when a SetRequest packet is received.
PDUTrace	Fired for every packet sent or received.
ReadyToSend	Fired when the component is ready to send data.
SetRequest	Fired when a SetRequest packet is received.
Trap	Fired when a SNMP trap packet is received.

Methods

DoEvents	Processes events from the internal message queue.
Reset	Clears the object arrays.
SendGetBulkRequest	Send a GetBulkRequest packet.
SendGetNextRequest	Send GetNextRequest packet.
SendGetRequest	Send GetRequest packet.
SendGetResponse	Send Get Response packet.
SendInformRequest	Send an InformRequest packet.
SendSetRequest	Send Set Request packet.
SendTrap	Send Trap packet.

SNMP Query - The simplest of SNMP applications: broadcasts a request on the LAN asking for system descriptions (1.3.6.1.2.1.1.1.0).

C++ Builder | Delphi | Java | Visual Basic | Visual C++ | Microsoft .NET

SNMP Agent - A simple SNMP agent that will respond and implement SNMP manager requests for system information. The Agent also shows how to broadcast traps.

Delphi | Visual Basic | Microsoft .NET

SNMP Manager - Shows how to create a basic SNMP manager that will find and query SNMP agents for their system information. The manager will also react to SNMP traps.

Delphi | Visual Basic | Microsoft .NET

2. เครื่องมือช่วยในการพัฒนาการแสดงผลในรูปแบบกราฟฟิค JClass



JClass helps you build Java applications quickly and easily, offering Java developers world-class versions of the components required by many standard applications, such as charts, tables, and reporting/printing.

Why re-invent the wheel? All JClass components are built with pride, have been fully tested and are ready to work

[JClass DesktopViews](#) Java class libraries give you J2SE components for rich-client GUI development.

[JClass ServerViews](#) offers thin-client components for J2EE application server environments.

JClass Nominated for Three JDJ Awards

Your peers, the readers of Java Developer's Journal, have nominated JClass for three awards in this year's Reader's Choice competition, including:



- Best Java Class Library
- Best Java Component
- Best Java Reporting Tool ([JClass ServerReport](#))

[Find out more.](#)

JClass ServerViews

[JClass ServerViews](#) is [JClass ServerChart](#) and [JClass ServerReport](#), pure J2EE components that generate interactive charts and PDF documents on the fly, bringing live professional data visualization to Web browsers. Read the ServerViews [datasheet](#).

JClass DesktopViews

[JClass DesktopViews](#), formerly the JClass Enterprise Suite, is a collection of components that give you the functionality required to build client-side applications with rich interface requirements. JClass DesktopViews includes both [JClass Chart](#) and [JClass LiveTable](#). Read the DesktopViews [datasheet](#).

Rich Java Clients

If your application needs more functionality than can be delivered through a Web browser, you may have already made the move to rich client development. Perhaps you're already using [JClass DesktopViews](#) to offer end-users better and faster data presentation.

With the extra functionality of rich clients comes the reality of application deployment. The initial installation often goes reasonably well, but the iterative nature of internal IT development means that a new version is ready on a weekly or monthly basis. After three or four cycles, you're looking for a deployment solution

That's where [Sitraka DeployDirector](#) comes in. It offers rich client deployment, class-level differencing for the leanest deployment possible, rollback, reporting, and email/wireless notification to deal with the unexpected. [Take the walkthrough](#).

Component-based Development

Components built in-house for a specific application tend to be used only once. When the next project comes along, the need for more components often accompanies it. Building these customized components can be time consuming and very costly given their short life span. Enter [JClass](#).

JClass components offer unrivaled [IDE, JDK and platform support](#), and include one year of premium technical support and free upgrades.

- [JClass Chart](#) - Embed the power of professional graphs and charts in your GUIs
- [JClass Chart 3D](#) - Interactive, three-dimensional data presentation
- [JClass LiveTable](#) - The essential Java grid for creating professional tables and forms
- [JClass Field](#) - Data validation for a range of popular data types
- [JClass PageLayout](#) - Professional printing & reporting power in Java
- [JClass Elements](#) - The Essential Set of GUI extensions and enhancements
- [JClass HiGrid](#) - The ideal front end for database applications

- [JClass DataSource](#) - A powerful tool for data access
- [JClass JarMaster](#) - The fastest way to package and manage your Java classes for rapid download
- [Gold Support with Subscription](#) - A year's worth of high-quality technical support from our Customer Support Engineers.

Features

- Integrated, high-level 100% Pure Java GUI JavaBeans
- Compatibility with any JavaBean-compliant IDE
- Connectivity to multiple levels of a master-detail relationship
- Robust database connectivity through JClass DataSource or through an IDE's data binding object
- Royalty-free
- Full documentation
- Bytecode and source code available.

JClass JDK Requirements and Supported Platforms

JClass components work with any platform that supports 100% Pure JavaBeans components and JFC/Swing 1.1. JClass products require the same operating system version on each platform as the JDK.

Platform support:

- [JClass DesktopViews 6.1](#)
- [JClass ServerViews 3.0](#)

The following table lists the supported platforms, and minimum JDK, web browser and Java IDE requirements for JClass DesktopViews 6.1.

Minimum JDK Requirements: ⁽¹⁾	6.1.0	JClass Chart 3D	
	(not including Chart 3D)	Java 2D	Java 3D

	JDK 1.2.2 (or higher)	JDK 1.2.2 (or higher)	JDK 1.3.1 Java 3D 1.2.1_04 OpenGL
Supported Platforms:			
MS Windows NT/2000/Me/XP/64	✓	✓	✓
Solaris 2.6, 2.7, 2.8, or 2.9	✓	✓	✓
HPUX 11	✓	✓	✓
IBM AIX 4.3.3 or 5.1	✓	✓	✓
RedHat Linux 7.x (Intel or 64)	✓	✓	✓
Supported Browsers:			
Netscape 6.1 or later is also supported, excluding the Java 3D version of JClass Chart 3D	n/a	n/a	n/a
with Java Plug-In	✓	✓	n/a
MS Internet Explorer 5 or 6	n/a	n/a	n/a
with Java Plug-In	✓	✓	n/a
Supported IDEs:			
Borland JBuilder 5 or higher	✓	✓	n/a
IBM VisualAge for Java 3.5 or higher	✓	✓ for 4.0 or later	n/a
Forte CE 3.0 or higher	✓	✓	n/a
IntelliJ IDEA 3.x	✓	✓	n/a
⁽¹⁾ We recommend using the most recent JDK version available for your platform.			

The following table lists the supported platforms and Java IDE requirements for JClass ServerViews 3.0:

Supported Server Platforms		Windows		Solaris			AIX		Linux	HPUX
		NT	2K	2.6	2.8	9.0	4.33	5.1	7.0	11
WebLogic	5.x	✓	✓	✓	✓					
	6.x	✓	✓	✓	✓					
	7.0	✓	✓		✓					✓
WebSphere	3.5.x	✓	✓	✓	✓		✓			
	4.x	✓	✓	✓	✓		✓			
VisualAge	4.0	✓	✓							
JRun	3.1	✓	✓	✓	✓		✓			
iPlanet	4.1	✓	✓	✓	✓		✓			
Tomcat with Apache server	latest								✓	
Tomcat	3.x	✓	✓	✓	✓	✓	✓	✓	✓	✓
Tomcat	4.x	✓	✓	✓	✓	✓	✓	✓	✓	✓

Supported Client Platforms	JClass ServerChart	JClass ServerReport
*Netscape 4.7, 6.x	✓	✓
*IE 4, 5, 6	✓	✓

*with SVG viewer plug-in 3.0, with Adobe Acrobat plug-in 5.0+

Supported IDEs	JClass ServerChart	JClass ServerReport
Borland JBuilder 5.x or higher	✓	✓
Forte 3.x	✓	✓
Visual Age 3.5.x	✓	✓
IntelliJ IDEA 3.x	✓	✓

ภาคผนวก ค

1. ระบบบัสสำหรับงานอุตสาหกรรม

เนื่องจากการขยายตัวของงานอัตโนมัติขึ้นมากในช่วง 10 ปีที่ผ่านมา ทำให้เทคโนโลยีการสื่อสารข้อมูลทางด้านอุตสาหกรรม (Industrial communication) มีความสำคัญมากขึ้น โดยจุดประสงค์เพื่อจัดการการสื่อสารข้อมูลที่มากและซับซ้อนของแต่ละลำดับชั้น ให้มีประสิทธิภาพมากที่สุด ซึ่งจากแนวโน้มการพัฒนาเทคโนโลยีที่ได้กล่าวมาแล้วนั้น คือ ได้มีการนำเทคโนโลยีระบบบัส (Bus system) มาใช้เพื่อให้บรรลุจุดประสงค์ที่ต้องการ

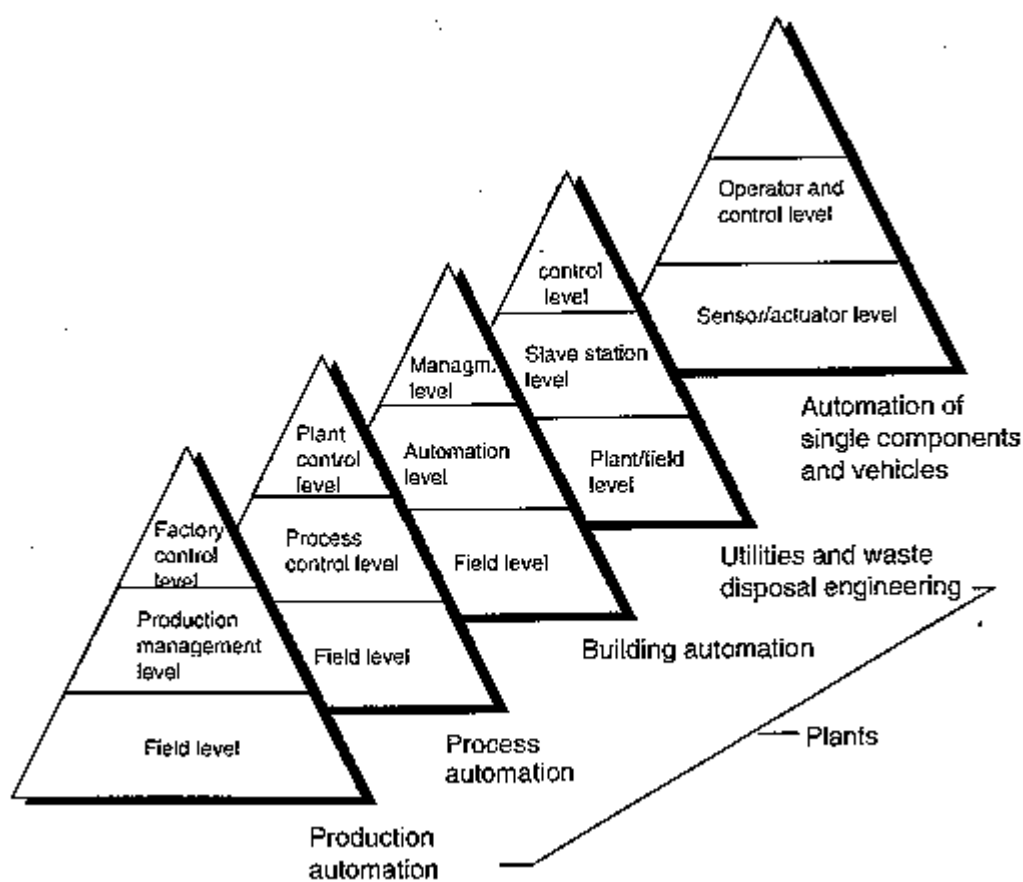
โดยทั่วไประบบบัสสำหรับงานอุตสาหกรรม (Industrial bus system) มีมากมายหลายชนิด ซึ่งในที่นี้จะขอจำแนกตามลำดับชั้นการติดต่อสื่อสารข้อมูลเป็น 3 ระดับ ดังนี้

1. ระดับ factory level ซึ่งจะครอบคลุมระดับอัตโนมัติระดับ Factory manage และ Coordinating
2. ระดับ Cell level ซึ่งจะครอบคลุมระดับอัตโนมัติระดับ System และ Control
3. ระดับ Field level ซึ่งจะครอบคลุมระดับอัตโนมัติระดับ Sensor actuator

ซึ่งเราสามารถสรุปเป็นตารางได้ดังนี้

ลำดับชั้นของ INDUSTRIAL COMMUNICATION			ชนิดของ INDUSTRIAL BUS ในท้องตลาด
Factory level			EtherNet TCP/IP
Control level			ARCNET, ControlNet, INTERBUS, PROFIBUS-FMS
Field level	Process Bus Network (Analog) (Up to 1000 Bytes)		FOUNDATION field bus, HART, INTERBUS, LON, PROFIBUS-FMS, PROFIBUS-PA
	Device Bus Network	Byte-wide data (8-256 Bytes)	BITBUS, CAN, CANopen, DeviceNet, FOUNDATION field bus, INTERBUS-S, PROFIBUS-DP, Smart Distributed System(SDS), Modbus RTU/ASII
	(Discrete)	Bit-wide data (น้อยกว่า 8 bits)	AS Interface, INTERBUS LOOP, Seriplex

ตาราง ค-1 จำแนกระบบบัสสำหรับงานอุตสาหกรรมตามลำดับทางการสื่อสาร



ลำดับการควบคุมระบบอัตโนมัติชั้นในงานต่างๆ

Factory level

EtherNet TCP/IP เป็นโปรโตคอลที่คนส่วนใหญ่จะคุ้นเคยดีในเรื่องของ Internet อุปกรณ์ทั้งซอฟต์แวร์ และฮาร์ดแวร์ก็เป็นสิ่งที่ทุกคนคุ้นเคยอยู่แล้วเพราะว่าใช้กันทั่วไปในสำนักงาน โดยทั่วไปแล้วเราจะใช้เทคโนโลยีอันนี้เพื่อเป็นการควบคุมระดับสูงสุดในโรงงาน

ประวัติของ TCP/IP นั้นมีกำเนิดมาจากระบบเครือข่าย ชื่อ ARPANET ในปี 1970 ซึ่งได้รับการสนับสนุน

การวิจัยจากกระทรวงกลาโหมประเทศสหรัฐอเมริกา ต่อมาได้มีการพัฒนาจนเป็นมาตรฐานในการรับ-ส่งข้อมูลผ่านระบบเครือข่ายอย่างเป็นทางการเมื่อวันที่ 1 มกราคม ค.ศ. 1983 ซึ่งปรากฏจำนวนระบบเครือข่าย และผู้ใช้เพิ่มขึ้นอย่างรวดเร็วจนเป็นที่รู้จัก ซึ่งถูกเรียกว่าเป็นระบบอินเทอร์เน็ต (Internet)

Control level

ARCNET (Attached Resource Computer Network)

ARCNET ถูกพัฒนาตั้งแต่ปี 1977 โดยบริษัท SMC ประเทศสหรัฐอเมริกา

Control Net ถูกพัฒนาโดยบริษัท Allen-Bradley ในปี 1995 เพื่อใช้ในงานควบคุมระดับ Control level

PROFIBUS-FMS (Profibus Fieldbus Message Specification) ถูกพัฒนาโดยบริษัท Siemens ประเทศเยอรมนี ในปี 1987 ในชื่อของ PROFIBUS (Process Fieldbus) และนำออกสู่ตลาดในปี 1992

- **Field level** ในงาน Process automation

FOUNDATION field bus ได้ถูกพัฒนามาบนพื้นฐานของมาตรฐาน IEC โดยได้รับการร่วมมือกันของ Noen World FIP และ ISP Foundation โดยจุดประสงค์เพื่อสร้างมาตรฐานฟิลด์บัสเพื่อให้สามารถใช้งานได้กับทุกยี่ห้อ

HART (Highway Addressable Remote Transducer) ถูกพัฒนาขึ้นในปี 1984 เป็นโปรโตคอลสำหรับอุปกรณ์ระดับฟิลด์ที่ใช้มาตรฐาน 4-20 mA

LON (Local Operating Network) เป็นระบบเน็ตเวิร์คสำหรับงานอัตโนมัติโดยใช้พื้นฐานของชิพ “neuron” ซึ่งถูกแนะนำออกสู่ตลาดในปี 1990 โดยบริษัท Echelon ของประเทศสหรัฐอเมริกา

PROFIBUS-PA (Profibus Process Automation) เป็น PROFIBUS ที่ใช้สำหรับงานควบคุมกระบวนการผลิต (Process control) โดยเฉพาะซึ่งจำเป็นต้องมีความปลอดภัยสูงมาก

World FIP (World Factory Instrumentation Protocol) เป็นมาตรฐานเปิด (Open system) ตามมาตรฐาน UTE 46 ซึ่งถูกเสนอให้เป็นมาตรฐานตาม IEC และ ISA สมาคมผู้ใช้เทคโนโลยี FIP ถูกตั้งขึ้นในปี 1993 โดยแกนนำหลัก ๆ คือ บริษัท Honeywell และ Bailey จากนั้นอีก 2 ปี (ค.ศ. 1994) ได้รวมตัวกับสมาคมผู้ใช้เทคโนโลยี ISP (Interoperable System Project) โดยมีแกนนำหลัก ๆ คือ บริษัท Fisher – Rosemount, Siemens และ Yokogawa) รวมกันเป็นสมาคมฟิลด์บัส (Fieldbus Foundation)

- **Field level** ในงาน Factory automation

Byte data

BITBUS ถูกพัฒนาโดยบริษัท INTEL และถูกใช้งานตั้งแต่ปี 1984 เป็นไปตามมาตรฐาน IEEE

CAN (Controller Area Network) เริ่มต้นในปี 1980 โดยบริษัท Bosch เพื่อลดปริมาณสายไฟ ในรถยนต์ซีดีส-เบนซ์ จากนั้นโปรโตคอลนี้ได้ถูกพัฒนาเพื่อการใช้งานที่มากขึ้น

CAN open อยู่ในตระกูล CAN ถูกพัฒนาโดยองค์การ CiA (CAN in Automation) ในปี 1993 เพื่อพัฒนาให้โปรโตคอล CAN มีความสามารถมากขึ้น จุดเด่นของ CAN open อยู่ต่างกับระบบ Fielbbus อื่น ๆ ก็คือ แต่ละ โหนด (node) สามารถติดต่อกับ โหนดโดยตรงได้โดยไม่ต้องติดต่อผ่านตัวแม่ Master เป็นการลดจำนวนการติดต่อสื่อสารได้มาก

Device Net เป็นมาตรฐานที่ถูกพัฒนามาจาก CAN โดยบริษัท Allen-Bradley ในปี 1994

INTERBUS-S ถูกพัฒนาโดยบริษัท Phoenix Contact ในปี 1984 เป็นระบบ Field bus ที่ได้รับความนิยมเนื่องจากความเร็ว การวิเคราะห์ จุดบกพร่อง การกำหนด address โดยอัตโนมัติ

PROFIBUS-DP (Profibus Distributed Periperal) เป็น PROFIBUS ที่ใช้สำหรับงานควบคุม เครื่องจักร (Factory automation) จากข้อมูลการตลาด PROFIBUS ถือได้ว่าเป็นผู้นำทางด้าน Field bus เลขที่ว่าได้เพราะมีคนนิยมใช้มากกว่าครึ่งหนึ่งของตลาด field bus

SDS (Smart Distributed System) ถูกพัฒนาโดยบริษัท Honeywell ซึ่งมีพื้นฐานมาจาก CAN

Bit data

As interface

INTERBUS LOOP เป็น INTERBUS ที่ใช้งานระดับ Actuator sensor เพื่อติดต่อสื่อสารข้อมูล ไม่กีดกัน

Seriplex เป็นระบบบัสที่ใช้ควบคุมอุปกรณ์ระดับ Actuator sensor ถูกพัฒนาโดยบริษัท APC (Automated Process Control)

2. The Internet Factory

ในหลายปีที่ผ่านมาอินเทอร์เน็ตได้เข้ามาปฏิวัติชีวิตประจำวันของพวกเรา และปัจจุบันอินเทอร์เน็ตก็เริ่มเข้ามามีบทบาทในการปฏิวัติโรงงานอุตสาหกรรมซึ่งจะทำให้การผลิตมีความยืดหยุ่นมากขึ้น ตลาดขึ้น และเชื่อมโยงกับสถานะที่แข่งขันกันมากขึ้น เทคโนโลยีอัตโนมัติภายในโรงงานจะต้องเชื่อมโยงกับห้องของกรรมการโรงงาน และเทคโนโลยีอินเทอร์เน็ตมาตรฐานคือคำตอบ

การประสานผลระหว่างออฟฟิศกับสายงานการผลิต

“กระบวนการอัตโนมัติที่รวดเร็วยิ่งขึ้นจากใบตอบรับการสั่งซื้อแบบออนไลน์(On line) จนถึงการส่งมอบของ” คือ เป้าหมายที่ถูกประกาศใช้ แต่ทว่าปัจจุบันในโลกของอิเล็กทรอนิกส์ กระบวนการหลัก 2

อย่างคือ การผลิต (manufacturing) และการจัดเก็บและส่งสินค้า (logistics) ยังคงแยกส่วนจากกันอยู่มาก ในขณะที่การสั่งซื้อนั้นเกิดขึ้นจากการคลิกเมาส์ (Mouse) โดยที่การจัดส่งของยังคงต้องใช้เวลาอยู่บ้าง อย่างไรก็ตามกลยุทธ์ e-business ที่ประสบความสำเร็จไม่ควรมุ่งประเด็นไปที่การขายเพียงอย่างเดียว แต่ควรคำนึงถึง e-manufacturing และ e-logistics ร่วมด้วย วิธีการที่คำนึงถึงธุรกิจในทุกระดับมีความจำเป็น ซึ่งได้แก่ ตลาดออนไลน์ การวางแผนทรัพยากร และระบบการบริหาร Supply Chain ระบบอัตโนมัติของการผลิตและการจัดเก็บและส่งสินค้า ดังนั้นออฟฟิศและสายงานการผลิตมีความจำเป็นที่จะต้องผสานกันเพื่อให้บรรลุผลได้ การวางแผนทรัพยากรของธุรกิจและระบบการจัดการ Supply Chain ควรได้ข้อมูลจริงจากสายงานการผลิตโดยใช้เทคโนโลยีอัตโนมัติ ซึ่งหมายความว่าอุปกรณ์สนามหรืออุปกรณ์หน้างาน เช่น เครื่องจักรต้องใช้

งานร่วมกับเว็บ(web) ได้ และคำตอบจากออฟฟิศมาตรฐานสามารถถูกเปลี่ยนแปลงได้ โดยโปรโตคอลเปิด อาทิเช่น TCP/IP และ HTTP สามารถประยุกต์ใช้ได้กับสภาพแวดล้อมแบบอัตโนมัติ Embedded web server สร้างความมั่นใจในแง่ของความง่ายและการเชื่อมต่อตรงเข้ากับอุปกรณ์ทั้งหมดซึ่งมี home page เป็นของตนเอง เบราร์เซอร์มาตรฐาน เช่น Microsoft's Internet Explorer หรือ Netscape's Navigator สามารถใช้งานเพื่อ Configure อุปกรณ์สนามได้อันส่งผลให้เกิดประโยชน์ขึ้นมากมายไม่จำเป็นที่จะต้องจับหรืออยู่ข้างอุปกรณ์เพื่อที่จะเปลี่ยนพารามิเตอร์ของมัน หรือ reconfigure อุปกรณ์ หรือทำการตรวจสอบข้อผิดพลาดของอุปกรณ์ สิ่งเหล่านี้สามารถกระทำได้จากสถานที่ต่าง ๆ ภายในโรงงาน หรือจากอีกโรงงานหนึ่ง หรือแม้กระทั่งจากอีกประเทศหนึ่ง ซึ่งเหมือนกับการเข้าถึง server ต่าง ๆ ซึ่งมี IP address เป็นของตนเองใน World Wide Web โดยอุปกรณ์สนามแต่ละตัวในโรงงานสามารถควบคุมได้ทางไกล (remote control) สิ่งเหล่านี้จะช่วยลดต้นทุนในการฝึกอบรม และลด down time นอกจากนี้อุปกรณ์สามารถถูกโปรแกรมให้ส่งอี-เมลล์ หรือข้อความ SMS แก่พนักงานในระดับปฏิบัติการได้ โดยใช้ XML หรือ extensible Markup Language จะทำให้ Internet Factory สามารถใช้งานได้เต็มที่ศักยภาพ XML นั้นมีข้อดีเหนือ HTML หรือสนับสนุนข้อมูลแบบ dynamic ในขณะที่ HTML ใช้เพื่อแสดงข้อมูลแบบ Static XML สามารถใช้ได้ในทุกระดับชั้นของธุรกิจ และระหว่างบริษัท XML เป็นภาษาของทางเลือกและสนับสนุนการแลกเปลี่ยนและประมวลผลข้อมูลแบบที่มีโครงสร้าง

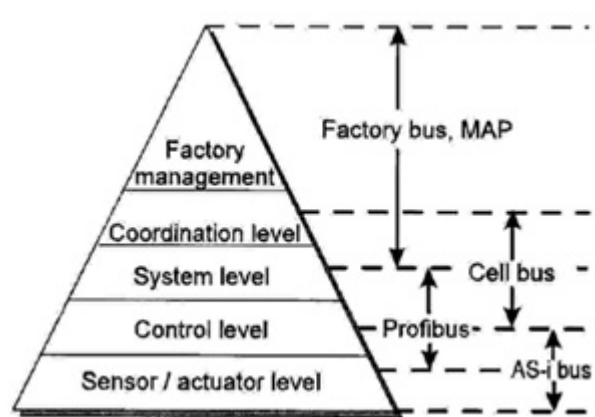
อินเทอร์เน็ตได้กลายเป็นปัจจัยหลักในการแข่งขันกันระหว่างบริษัท และเทคโนโลยี Web กำลังสร้างมาตรฐานใหม่ใน factory automation ความท้าทายจึงเกิดขึ้นกับผู้มีอำนาจในการตัดสินใจ คือการกำหนดกลยุทธ์รวมเพื่อใช้งานสิ่งเหล่านี้ให้เต็มศักยภาพ ซึ่งสามารถสรุปคร่าว ๆ ได้ 2 ขั้นตอน คือ

- Embedded web servers หรือการเข้าถึงผ่าน Internet สำหรับเครื่องจักรทั้งหมด และอุปกรณ์ทั้งหมด ข้อมูลในกระบวนการสามารถเข้าถึงได้ผ่าน Internet

การวางแผนทรัพยากรของธุรกิจและระบบการบริหาร supply chain ต้องเข้าถึงได้โดย web ดังนั้นกระบวนการธุรกิจระหว่างผู้ผลิต ผู้ร่วมถึง และลูกค้า สามารถที่จะบรรลุผลได้โดย internet

3. ลำดับชั้นของการควบคุมระบบอัตโนมัติ

คำว่า “Automation hierarchy” นั้นกล่าวถึงระดับของการควบคุมงานอัตโนมัติ พื้นที่ที่ใช้ความสามารถในการทำงานและทิศทางการไหลของข้อมูล ทั้งนี้เพื่อประโยชน์ในการจัดการข้อมูลที่ซับซ้อนของระบบอัตโนมัติ ซึ่ง Automation hierarchy จะถูกแบ่งเป็น 5 ระดับชั้นด้วยกันดังนี้



ลำดับชั้นของการควบคุมระบบอัตโนมัติ

ระดับที่ 1 : Factory management level เป็นระดับที่สูงสุดโดยมีความสำคัญในการตัดสินใจเกี่ยวกับการวางแผนและความคุมการผลิต (Planning and Production Control : PPC) ซึ่งจำเป็นที่จะต้องนำข้อมูลจากทุกฝ่าย เช่น ฝ่ายขาย ฝ่ายบัญชี ฝ่ายต้นทุน ฝ่ายวัสดุ เป็นต้น มาทำการประมวลผลเพื่อการตัดสินใจ

ระดับที่ 2 : Coordinating level เป็นระดับที่มีหน้าที่รับคำสั่งมาจากระดับที่ 1 จากนั้นก็จะทำการแจกจ่ายงานไปยังหน่วยการผลิต (work cell) เช่น การส่งงานไปที่หน่วยประกอบ (Assembly cell) หน่วยสต็อก (Store cell) หน่วยการขึ้นรูป (Machine tool cell) จากนั้นก็จะมีการรายงานผลไปยังระดับที่ 1 เพื่อใช้ในการประมวลผลต่อไป

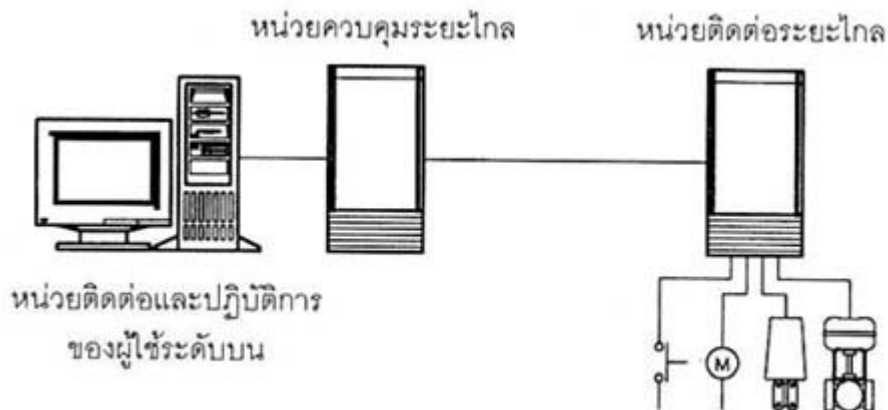
ระดับที่ 3 : System level เป็นระดับหน่วยการผลิต (Cell level) ซึ่งจะทำหน้าที่ดูแลหน่วยการผลิตนั้น ๆ ในทุก ๆ เรื่องอย่างเช่น การกำหนดขั้นตอนการผลิตการซ่อมบำรุง (Maintenance) การวิเคราะห์งาน (Diagnostic) การควบคุมคุณภาพ

ระดับที่ 4 : Control level เป็นระดับของคอนโทรลเลอร์เช่น RC (Robotic Controller), CNC , PLC (Programmable logic Controller)

ระดับที่ 5 : Sensor actuator level เป็นระดับของอุปกรณ์ทำงานและเซนเซอร์ซึ่งเป็นระดับล่างสุด

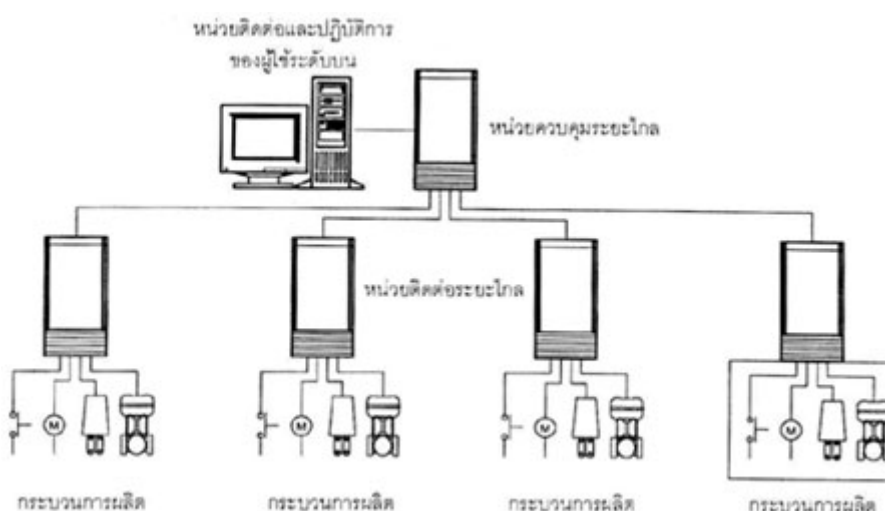
4. SCADA

สกาดา (SCADA – Supervisory Control and Data Acquisition) คือระบบเครื่องมืออัตโนมัติสำหรับตรวจสอบเก็บรวบรวมข้อมูลและบริหารระบบควบคุมของกระบวนการผลิตภายในโรงงานอุตสาหกรรม สกาดาประกอบด้วยส่วนประกอบหลักคือ หน่วยติดต่อและปฏิบัติการของผู้ใช้ระดับบน หน่วยควบคุมระยะไกล และหน่วยติดต่อระยะไกล (ดังรูปที่ 1)



รูปที่ 1 องค์ประกอบของระบบสกาดา

ผู้ใช้สามารถตรวจสอบและควบคุมกระบวนการผลิตภายในโรงงานอุตสาหกรรมเป็นระยะทางไกลได้โดย หน่วยติดต่อและปฏิบัติการของผู้ใช้ระดับบนเป็นเครื่องมือปฏิบัติการของผู้ใช้สำหรับตรวจสอบและควบคุม กระบวนการผลิตเชื่อมต่อกับหน่วยควบคุมระยะไกล หน่วยควบคุมระยะไกลติดต่อกับหน่วยติดต่อระยะไกลโดยการสื่อสารข้อมูลแบบดิจิทัลทางระบบเครือข่ายคมนาคม และหน่วยติดต่อระยะไกลเป็นเครื่องมือเชื่อมต่อกับกระบวนการผลิต ประกอบด้วย หน่วยรับสัญญาณ และส่งสัญญาณของสัญญาณชนิดแอนะล็อก และสัญญาณชนิดดิจิทัล



การติดตั้งสกาดาสำหรับตรวจสอบ เก็บรวบรวมข้อมูล และบริหารระบบควบคุม



สกาดาเหมาะสมกับงานประเภทใด

สกาดาเหมาะสำหรับการตรวจสอบ และเก็บรวบรวมข้อมูลของกระบวนการผลิต และการบริหารระบบควบคุมของกลุ่มโรงงานอุตสาหกรรมขนาดใหญ่บริเวณกระบวนการผลิตครอบคลุมพื้นที่กว้าง หรือโรงงานอุตสาหกรรมมีกระบวนการผลิตอิสระติดตั้งกระจายทั่วบริเวณพื้นที่การผลิต รวมถึงระบบสาธารณูปโภคต่าง ๆ เช่น ระบบจ่ายไฟฟ้าและน้ำประปา ระบบท่อส่งก๊าซและน้ำมัน สกาดาไม่เหมาะสำหรับการควบคุมกระบวนการชนิดสัญญาณต่อเนื่องและการควบคุมกระบวนการแบบติดและดับของกระบวนการผลิตทั่วไป เนื่องจากสกาดาสามารถคำนวณและประมวลผลข้อมูลโดยหน่วยประมวลผลภายในหน่วยติดต่อกันและปฏิบัติการของผู้ใช้ระดับบนซึ่งไม่มีหน้าท ี่สำหรับการควบคุมกระบวนการผลิตโดยตรงการรับสัญญาณวัดและส่งสัญญาณควบคุมระหว่างสกาดากับกระบวนการผลิตต้องผ่านการเชื่อมต่อ ของระบบเครือข่ายทำให้การควบคุมกระบวนการผลิตของสกาดาไม่มีประสิทธิภาพเช่นเดียวกับระบบควบคุมแบบอื่น

บรรณานุกรม

- [1] Gilbert Held: "LAN Management with SNMP and RMON", Jonh Wiley, 1996
- [2] สุรศักดิ์ สงวนพงษ์: "สถาปัตยกรรมและโปรโตคอลทีซีพี/ไอพี", ซีเอ็ด, 2002
- [3] สุวัฒน์ ปุณณชัยยะ, ตัน ตันท์สุทธีวงศ์, สุพจน์ ปุณณชัยชนะ, "เปิดโลกของ TCP/IP และโปรโตคอลของ อินเทอร์เน็ต", โปรวิชั่น, 2543
- [4] Ivar Jacobson, Grady Booch, James Rumbaugh: " The Unified Software Development Process", Addison Wesley ,1999
- [5] William Stallings: "SNMP,SNMPv2, and RMON Practical Network Management" ,Addison Wesley ,1996
- [6] Gilbert Held: "LAN Management with SNMP and RMON", John Wiley, 1996
- [7] Edward Strut: "Network Management Concepts and Tools", Chapman & Hall, 1994
- [8] อภินันท์ อุณาภูล: "Object-Oriented analysis and design" ,แผนกตำรา วิศวกรรมศาสตร์ สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง, 2543
- [9] รัชฎ์ กิจระภูมิ. 2544. การใช้งาน Microsoft SQL Server 2000 Step by Step. กรุงเทพฯ : สามย่าน.COM

เว็บไซต์อ้างอิง

<http://www.nsoftware.com>
<http://www.adventnet.com>
<http://www.java.sun.com>
<http://www.microsoft.com/java>
<http://nms.estig.ipb.pt>
<http://www.agilent.com/comms/oss>
<http://netman.cit.buffalo.edu>
http://www.nsbasic.com/ce/info/nsbce/BigRedToolbox/ctl_SNMP.htm
<http://www.logisoftar.com/ProductsSnmpTkVb.htm>
<http://www.dart.com/powertcp/snmp.asp>
<http://www.hallogram.com/snmpbuilder>
http://www.hughes.com.au/library/lite/modules/mod_snmp/
<http://www.et.put.poznan.pl/snmp/main/mainmenu.html>
<http://www.snmplink.org>
<http://www.simpleweb.org>

<http://www.sercomm.com.tw/ups.htm>

<http://www.ece.arizona.edu/department/projects/presentation/>

<http://www.nmops.org/>

<http://www.oreilly.com/catalog/esnmp/>

<http://user.gru.net/frenchkc/snmp.html>

<http://www.ibr.cs.tu-bs.de/projects/snmpv3>

<http://silver.he.net/~rrg/snmpworld.htm>

<http://www.networksorcery.com/enp/protocol/snmp.htm>